

Freestanding constexpr containers and constexpr exception types

Document number: P3295R3

Date: 2025-03-25

Reply-to: Ben Craig <ben dot craig at gmail dot com>

Audience: Library Working Group

Changes from previous revisions

R3

- Making all of vector and string freestanding-constexpr.
- Removing constexpr <exceptions> and <stdexcept> parts, as those are already in the standard.
- Bumping feature test macros for memory and string_view headers.

R2

- Better aligning with P3160 and P3418's allocator design.
- Mentioning P3421: Consteval Destructors

R1

- Discussing implementation experience.
- Adding all stdexcept exceptions.
- Discussing default constructed containers and operator delete.
- Discussing methods remaining constexpr.
- Discussing allocator requirements difficulty
- Discussing merge conflicts with in-flight constexpr exception and allocator papers.
- Wording improvements.

R0

First revision!

Introduction

This proposal helps enable [P2996 reflection](#) and more advanced metaprogramming for freestanding environments. Specifically, this allows `std::vector`, `std::string`, and `std::allocator` to be used in constant evaluated contexts in freestanding.

As a first approximation, this makes the `vector` and `string` facilities constexpr in freestanding. Freestanding implementations that know more about their targets are permitted to provide the full constexpr facilities if they wish.

This paper is attempting to modify the minimum set of facilities possible to freestanding in order to enable reflection, plus some additional exception types that come along for "free".

Motivation and Scope

Freestanding applications are often unable to allocate memory or throw exceptions at runtime. No such restriction exists while building the application though.

[Reflection in P2996R2](#) contains many consteval functions that return `vector<meta::info>`, such as `members_of`. Those facilities are valuable in freestanding and hosted alike.

This paper would "only" make `vector` and `allocator` consteval on freestanding if it reasonably could as a means to limit scope, but `vector` has functions that throw `length_error` and `out_of_range`. `length_error` and `out_of_range` are not in freestanding prior to this paper, so mentioning them renders the program ill-formed, even in a constant evaluation context. This paper therefore makes those exception types consteval on freestanding.

Similarly, `length_error` and `out_of_range` mention `string`, which is also not yet in freestanding. Therefore the scope expands to `string`. Fortunately, `char_traits` and most of `string_view` are already freestanding.

All the `<stdexcept>` exceptions are receiving the same consteval treatment, since the classes are largely identical other than the class names.

The *freestanding-deleted* methods of `string_view` will need to be made consteval on freestanding in order to support strings use cases.

freestanding-consteval additions:

- `allocator`
- `string_view` freestanding-deleted members
- `string`
- `vector`

- `logic_error`
- `domain_error`
- `invalid_argument`
- `length_error`
- `out_of_range`
- `runtime_error`
- `range_error`
- `overflow_error`
- `underflow_error`

Design Considerations

Freestanding all the constexpr things?

In [P2268 Freestanding Roadmap](#), a strategy is discussed of making everything that is constexpr in the standard library available at compile time via consteval, and those facilities that are also suitable at runtime to be marked constexpr and `// freestanding`. That is still the desired goal. Reaching that goal requires a lot of research and testing. There are likely other, non-constexpr facilities that the constexpr facilities rely on that would need to be audited as well.

Destructors cannot currently be marked as `constexpr`. virtual functions that override `constexpr` methods cannot currently be marked as `constexpr`. Those restrictions also prevent us from blanket `constexpr`'ing all the `constexpr` facilities in the standard library. This paper has chosen to prioritize the reflection subset of that work, rather than block progress on freestanding reflection.

Performance regression when porting from freestanding to hosted

`constexpr` functions are never evaluated during runtime, but `constexpr` functions are sometime evaluated during runtime. A freestanding library using a `constexpr` facility may end up "downgrading" some code from compile time to runtime during a port to a hosted environment due to the change to `constexpr`. This can result in worse performance due to the runtime function calls and allocations that were previously compile time function calls and allocations.

In [this unoptimized example](#), we get runtime calls to an allocating constructor and deallocating destructor in the `constexpr` version compared to the `constexpr` version. For this example, the difference disappears when optimizations are enabled, but that won't always be the case.

Non-transient `constexpr` allocations

[Non-transient allocation with vector and basic_string](#) is attempting to make it possible to allow constant evaluated vectors and strings persist into runtime. I'd like to avoid causing issues for non-transient allocation papers.

We can't rely on the future success of P3544 though. As a result this paper is making the entirety of `vector` and `basic_string` freestanding-`constexpr`. One non-transient allocation is here, there will be a desire to make non-throwing `const` methods `constexpr` on freestanding, rather than freestanding-`constexpr`. This would make it possible to observe the values in containers with non-transient allocations possible at runtime. We would still be able to avoid allocations and exceptions at runtime, while also allowing the constant compile time object to be observed.

Instead, we could have marked the functions as `constexpr` in this paper. One unfortunate aspect of doing so is that it would make `operator[]`, `front()`, and `back()` as potentially callable at runtime, even though the only possible runtime containers are empty. LEWG should discuss whether we should prioritize future-proofing the containers or mistake-proofing them. For now, the paper has chosen to mistake-proof.

Freestanding operator `delete` and default constructed containers

Even when marked as `constexpr`, a default constructed `std::string` or `std::vector` can persist into runtime. The destructor for such an object can be executed at runtime. That destructor will run `operator delete` on a `nullptr`.

Freestanding implementations may not have a heap [\[new.delete.general\]#2](#). Conforming implementations aren't required to have a useable `operator new`, but they are required to have an `operator delete` that can handle a `nullptr`. That means that it is acceptable (though unfortunate) that an `operator delete` "escapes" into runtime. In practice, I don't think this will be a big problem, as there isn't much that can be done with a default constructed runtime container where all the mutating operations are `constexpr`.

Ideally, this scenario would be ill-formed. The author has proposed [P3421: Consteval Destructors](#) which would improve this situation. `constexpr` destructors aren't a requirement

for this paper though.

Allocator requirements

Is `std::allocator` still an allocator if it can only allocate and deallocate at compile time?

In practice, this seems to work fine in code. So I'm leaving the requirements alone. This paper doesn't make any changes to the named requirements for allocators or `allocator_traits`.

Maybe in the future the standard will be more explicit about when library facilities need to be available at compile time vs. run time.

Experience

This proposal has been implemented in the Microsoft STL implementation, targeting the Microsoft Windows kernel. The binaries were compiled with clang-cl, as Microsoft's cl doesn't yet implement [P2564: consteval needs to propagate up](#). `constexpr` propagating up was necessary for compressed pair implementations. Some changes had to be made to `vcruntime` to move some `std::exception` code inline.

The `libc++` test suite was used to verify the changes. I modified the tests to verify the compile-time paths (if it wasn't doing so already), and remove the run-time paths. I would expect production implementations to do this with an `#ifdef` to remove the run-time paths when testing the freestanding implementation.

I will note that the Microsoft Windows kernel implementation does not support C++ exceptions. As a result, the tests did not exercise any throwing paths.

Merge Conflicts

Allocators

[P3160R2 "An Allocator-Aware `inplace\_vector`"](#) and [P3418R0 Wording for P3160R2 `inplace\_vector`'s "construct/destroy" alternative](#) need to make allocators freestanding, since `inplace_vector` is already freestanding.

`inplace_vector` doesn't directly need the `allocate` and `deallocate` methods in the allocator. This paper needs those methods, but only during constant evaluation.

For allocator wording ([`allocator.requirements.general`] and [`allocator.traits.memgers`]), this paper's wording matches draft versions of P3160. This paper's freestanding-`constexpr` changes in [`default.allocator`] use a different wording technique than the `inplace_vector` papers, but the intent is the same.

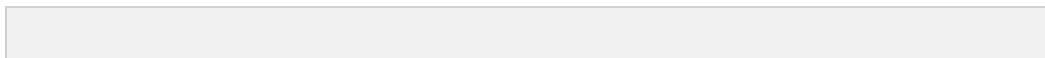
Wording

This paper's wording is based on the working draft N5008.

Changes in [\[functions.within.classes\]](#)

Instructions to the editor:

Move [`functions.within.classes`] below [`freestanding.item`]. This is so we can use the "freestanding `constexpr` classes" term after it is defined. Then modify [`functions.within.classes`] as follows:



- ¹ For the sake of exposition, [\[support\]](#) through [\[exec\]](#) and [\[depr\]](#) do not describe copy/move constructors, assignment operators, or (non-virtual) destructors with the same apparent semantics as those that can be generated by default ([\[class.-copy.ctor\]](#), [\[class.copy.assign\]](#), [\[class.dtor\]](#)). It is unspecified whether the implementation provides explicit definitions for such member function signatures, or for virtual destructors that can be generated by default. On freestanding implementations, it is implementation defined whether the copy/move constructors and assignment operators of freestanding consteval classes are immediate functions or constexpr functions.

Changes in [\[freestanding.item\]](#)

Instructions to the editor:

Please make the following edits to [\[freestanding.item\]](#):

- ³ Function declarations and function template declarations followed by a comment that include *freestanding-deleted* are *freestanding deleted functions*. On freestanding implementations, it is implementation-defined whether each entity introduced by a freestanding deleted function is a deleted function ([\[dcl.fct.def.delete\]](#)) or whether the requirements are the same as the corresponding requirements for a hosted implementation.

[*Note 1*: Deleted definitions reduce the chance of overload resolution silently changing when migrating from a freestanding implementation to a hosted implementation. — *end note*]

[*Example 1*:

```
double abs(double j);           // freestanding-deleted
```

— *end example*]

???a Function declarations and function template declarations followed by a comment that includes *freestanding-constexpr* are *freestanding constexpr functions*. On freestanding implementations, it is implementation-defined whether each entity introduced by a freestanding constexpr function is an immediate function ([\[expr.const\]](#)) or whether the requirements are the same as the corresponding requirements for a hosted implementation.

???b Class declarations and class template declarations followed by a comment that includes *freestanding-constexpr* are *freestanding constexpr classes*.

- ⁴ A declaration in a synopsis is a freestanding item if

(4.1) — it is followed by a comment that includes *freestanding*,

(4.2) — it is followed by a comment that includes *freestanding-deleted*, or

(4.???) **it is followed by a comment that includes *freestanding-constexpr*, or**

(4.3) — the header synopsis begins with a comment that includes *freestanding* and the declaration is not followed by a comment that includes *hosted*.

[*Note 2*: Declarations followed by *hosted* in freestanding headers are not freestanding items. As a result, looking up the name of such functions can vary between hosted and freestanding implementations. — *end note*]

[*Example 2*:

```
// all freestanding
namespace std {

— end example]

/*...*/
```

7 [Note 3: Freestanding annotations follow some additional exposition conventions that do not impose any additional normative requirements. Header synopses that begin with a comment containing "all freestanding" contain no hosted items, **and no freestanding deleted functions, no freestanding consteval functions, and no freestanding consteval classes**. Header synopses that begin with a comment containing "mostly freestanding" contain at least one hosted item, **or freestanding deleted function, freestanding consteval function, or freestanding consteval class**. Classes and class templates followed by a comment containing "partially freestanding" contain at least one hosted item, **or freestanding deleted function, or freestanding consteval function**. — end note]

Changes in [compliance]

Drafting note: <string> and <memory> are already freestanding headers.
Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[...]	[...]	[...]
?? [std.exceptions]	Exception classes	<stdexcept>
[...]	[...]	[...]
?? [vector.syn]	Header <vector> synopsis	<vector>
[...]	[...]	[...]

Changes in [version.syn]

Instructions to the editor:
Add the following macros to [version.syn]:

```
#define cpp_lib_freestanding_consteval_allocator 20XXXXL // freestanding, a
#define cpp_lib_freestanding_consteval_string 20XXXXL // freestanding, also
#define cpp_lib_freestanding_stdexcept 20XXXXL // freestanding, also in <st
#define cpp_lib_freestanding_vector 20XXXXL // freestanding, also in <vecto
```

Bump the constants on the following macros:

```
#define __cpp_lib_freestanding_memory 202502L 20XXXXL // freesta
/*...*/
#define __cpp_lib_freestanding_string_view 202311L 20XXXXL // freesta
```

Changes in [stdexcept.syn]

Instructions to the editor:

Please make the following edits to [\[stdexcept.syn\]](#):

```
namespace std {  
    class logic_error; // freestanding-consteval  
    class domain_error; // freestanding-consteval  
    class invalid_argument; // freestanding-consteval  
    class length_error; // freestanding-consteval  
    class out_of_range; // freestanding-consteval  
    class runtime_error; // freestanding-consteval  
    class range_error; // freestanding-consteval  
    class overflow_error; // freestanding-consteval  
    class underflow_error; // freestanding-consteval  
}
```

Changes in [\[logic.error\]](#)

Instructions to the editor:

Please make the following edits to [\[logic.error\]](#):

```
namespace std {  
    class logic_error : public exception {  
    public:  
        constexpr explicit logic_error(const string& what_arg); // freestanding-consteval  
        constexpr explicit logic_error(const char* what_arg); // freestanding-consteval  
    };  
}
```

Changes in [\[domain.error\]](#)

Instructions to the editor:

Please make the following edits to [\[domain.error\]](#):

```
namespace std {  
    class domain_error : public logic_error {  
    public:  
        constexpr explicit domain_error(const string& what_arg); // freestanding-consteval  
        constexpr explicit domain_error(const char* what_arg); // freestanding-consteval  
    };  
}
```

Changes in [\[invalid.argument\]](#)

Instructions to the editor:

Please make the following edits to [\[invalid.argument\]](#):

```
namespace std {  
    class invalid_argument : public logic_error {  
    public:
```

```
constexpr explicit invalid_argument(const string& what_arg); // freestanding-  
constexpr explicit invalid_argument(const char* what_arg); // freestanding-  
};
```

Changes in [\[length.error\]](#)

Instructions to the editor:

Please make the following edits to [\[length.error\]](#):

```
namespace std {  
    class length_error : public logic_error {  
    public:  
        constexpr explicit length_error(const string& what_arg); // freestanding-co  
        constexpr explicit length_error(const char* what_arg); // freestanding-consi  
    };  
}
```

Changes in [\[out.of.range\]](#)

Instructions to the editor:

Please make the following edits to [\[out.of.range\]](#):

```
namespace std {  
    class out_of_range : public logic_error {  
    public:  
        constexpr explicit out_of_range(const string& what_arg); // freestanding-co  
        constexpr explicit out_of_range(const char* what_arg); // freestanding-consi  
    };  
}
```

Changes in [\[runtime.error\]](#)

Instructions to the editor:

Please make the following edits to [\[runtime.error\]](#):

```
namespace std {  
    class runtime_error : public logic_error {  
    public:  
        constexpr explicit runtime_error(const string& what_arg); // freestanding-co  
        constexpr explicit runtime_error(const char* what_arg); // freestanding-con  
    };  
}
```

Changes in [\[range.error\]](#)

Instructions to the editor:

Please make the following edits to [\[range.error\]](#):

```

namespace std {
    class range_error : public logic_error {
    public:
        constexpr explicit range_error(const string& what_arg); // freestanding-con
        constexpr explicit range_error(const char* what_arg); // freestanding-conste
    };
}

```

Changes in [\[overflow.error\]](#)

Instructions to the editor:

Please make the following edits to [\[overflow.error\]](#):

```

namespace std {
    class overflow_error : public logic_error {
    public:
        constexpr explicit overflow_error(const string& what_arg); // freestanding-
        constexpr explicit overflow_error(const char* what_arg); // freestanding-co
    };
}

```

Changes in [\[underflow.error\]](#)

Instructions to the editor:

Please make the following edits to [\[underflow.error\]](#):

```

namespace std {
    class underflow_error : public logic_error {
    public:
        constexpr explicit underflow_error(const string& what_arg); // freestanding
        constexpr explicit underflow_error(const char* what_arg); // freestanding-c
    };
}

```

Changes in [\[memory.syn\]](#)

Instructions to the editor:

Please make the following edit to [\[memory.syn\]](#):

```

// [default allocator], the default allocator
template<class T> class allocator; // partially freestanding
template<class T, class U>
    constexpr bool operator==(const allocator<T>&, const allocator<U>&) noexcept

```

Changes in [\[default.allocator\]](#)

Instructions to the editor:

Please make the following edit to [\[default allocator\]](#):

```
namespace std {
    template<class T> class allocator {
    public:
        using value_type                = T;
        using size_type                 = size_t;
        using difference_type           = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;

        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
        constexpr ~allocator();
        constexpr allocator& operator=(const allocator&) = default;

        constexpr T* allocate(size_t n); // freestanding-constexpr
        constexpr allocation_result<T*> allocate_at_least(size_t n); // freestanding-constexpr
        constexpr void deallocate(T* p, size_t n); // freestanding-constexpr
    };
};
```

Changes in [\[string.view.template.general\]](#)

Instructions to the editor:

Please make the following edit to [\[string.view.template.general\]](#):

```
// [string.view.access], element access
constexpr const_reference operator[](size_type pos) const;
constexpr const_reference at(size_type pos) const; // free
constexpr const_reference front() const;
constexpr const_reference back() const;
constexpr const_pointer data() const noexcept;

// [string.view.modifiers], modifiers
constexpr void remove_prefix(size_type n);
constexpr void remove_suffix(size_type n);
constexpr void swap(basic_string_view& s) noexcept;

// [string.view.ops], string operations
constexpr size_type copy(charT* s, size_type n,
                        size_type pos = 0) const; // free

constexpr basic_string_view substr(size_type pos = 0,
                                size_type n = npos) const; // free

constexpr int compare(basic_string_view s) const noexcept;
constexpr int compare(size_type pos1, size_type n1,
                    basic_string_view s) const; // free
constexpr int compare(size_type pos1, size_type n1, basic_string_view s,
```

```

        size_type pos2, size_type n2) const; // free
constexpr int compare(const charT* s) const;
constexpr int compare(size_type pos1, size_type n1,
        const charT* s) const; // free
constexpr int compare(size_type pos1, size_type n1, const charT* s,
        ...

```

Changes in [\[string.syn\]](#)

Instructions to the editor:

Please make the following edits to [\[string.syn\]](#):

```

// \[basic.string\], basic_string
template<class charT, class traits = char_traits<charT>, class Allocator =
    class basic_string; // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
        const basic_string<charT, traits, Allocator>& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
        const basic_string<charT, traits, Allocator>& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,
        basic_string<charT, traits, Allocator>&& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(basic_string<charT, traits, Allocator>&& lhs,
        basic_string<charT, traits, Allocator>&& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(const charT* lhs,
        const basic_string<charT, traits, Allocator>& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(const charT* lhs,
        basic_string<charT, traits, Allocator>&& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
        const basic_string<charT, traits, Allocator>& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(charT lhs,
        basic_string<charT, traits, Allocator>&& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
    operator+(const basic_string<charT, traits, Allocator>& lhs,

```

```

        const charT* rhs); // freestanding-constexpr
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
        const charT* rhs); // freestanding-constexpr
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
        charT rhs); // freestanding-constexpr
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
        charT rhs); // freestanding-constexpr
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
        type_identity_t<basic_string_view<charT, traits>> rhs); // free
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
        type_identity_t<basic_string_view<charT, traits>> rhs); // free
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
        const basic_string<charT, traits, Allocator>& rhs); // freestanding-constexpr
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
        basic_string<charT, traits, Allocator>&& rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr bool
operator==(const basic_string<charT, traits, Allocator>& lhs,
        const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr bool operator==(const basic_string<charT, traits, Allocator>& lhs,
        const charT* rhs); // freestanding-constexpr

template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
        const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
        const charT* rhs); // freestanding-constexpr

// [string.special], swap
template<class charT, class traits, class Allocator>
constexpr void
swap(basic_string<charT, traits, Allocator>& lhs,
        basic_string<charT, traits, Allocator>& rhs)
noexcept(noexcept(lhs.swap(rhs))); // freestanding-constexpr

```

```

// [string.io], inserters and extractors
template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is,
                    basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,
                    const basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        getline(basic_istream<charT, traits>& is,
                 basic_string<charT, traits, Allocator>& str,
                 charT delim);
template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        getline(basic_istream<charT, traits>&& is,
                 basic_string<charT, traits, Allocator>& str,
                 charT delim);
template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        getline(basic_istream<charT, traits>& is,
                 basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
    basic_istream<charT, traits>&
        getline(basic_istream<charT, traits>&& is,
                 basic_string<charT, traits, Allocator>& str);

// [string.erase], erasure
template<class charT, class traits, class Allocator, class U = charT>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
        erase(basic_string<charT, traits, Allocator>& c, const U& value); // free
template<class charT, class traits, class Allocator, class Predicate>
    constexpr typename basic_string<charT, traits, Allocator>::size_type
        erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred); //.

// basic_string typedef-names
using string      = basic_string<char>; // freestanding
using u8string    = basic_string<char8_t>; // freestanding
using u16string   = basic_string<char16_t>; // freestanding
using u32string   = basic_string<char32_t>; // freestanding
using wstring     = basic_string<wchar_t>; // freestanding
/* ... */
inline namespace literals {
    inline namespace string_literals {
        // [basic.string.literals], suffix for basic_string literals
        constexpr string      operator""s(const char* str, size_t len); // freestanding
        constexpr u8string    operator""s(const char8_t* str, size_t len); // freestanding
        constexpr u16string   operator""s(const char16_t* str, size_t len); // freestanding
        constexpr u32string   operator""s(const char32_t* str, size_t len); // freestanding
        constexpr wstring     operator""s(const wchar_t* str, size_t len); // freestanding
    }
}

```

```
}
```

Changes in [\[basic.string\]](#)

Instructions to the editor:

Please add the following paragraph to the end of [\[basic.string\]](#):

? Other than the destructor, all member functions and member function templates in the class template basic_string are freestanding consteval functions ([freestanding.item]).

Changes in [\[vector.syn\]](#)

Instructions to the editor:

Please make the following edits to [\[vector.syn\]](#):

```
// [vector], class template vector
template<class T, class Allocator = allocator<T>> class vector; // freestanding

template<class T, class Allocator>
    constexpr bool operator==(const vector<T, Allocator>& x, const vector<T,
template<class T, class Allocator>
    constexpr synth-three-way-result<T> operator<=>(const vector<T, Allocator
                                                const vector<T, Allocator

template<class T, class Allocator>
    constexpr void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
        noexcept(noexcept(x.swap(y))); // freestanding-consteval

// [vector.erasure], erasure
template<class T, class Allocator, class U = T>
    constexpr typename vector<T, Allocator>::size_type
        erase(vector<T, Allocator>& c, const U& value); // freestanding-consteval
template<class T, class Allocator, class Predicate>
    constexpr typename vector<T, Allocator>::size_type
        erase_if(vector<T, Allocator>& c, Predicate pred); // freestanding-consteval

namespace pmr {
    template<class T>
        using vector = std::vector<T, polymorphic_allocator<T>>;
}

// [vector.bool], specialization of vector for bool
// [vector.bool.pspc], partial class template specialization vector<bool, Allocator>
template<class Allocator>
    class vector<bool, Allocator>; // freestanding-consteval
```

Changes in [\[vector\]](#)

Instructions to the editor:

Please add the following paragraph to the end of [\[vector\]](#):

? Other than the destructor, all member functions and member function templates in the class template `vector` are freestanding consteval functions ([freestanding.item]).

Changes in [\[vector.bool\]](#)

Instructions to the editor:

Please add the following paragraph to the end of [\[vector.bool\]](#):

? Other than the destructor, all member functions and member function templates in `vector<bool>` are freestanding consteval functions ([freestanding.item]).

Disclaimer

The opinions in this paper are my own and do not reflect the views or positions of my employer, CrowdStrike, Inc.